



Road Reorganization (roads)

L'armée romaine a récemment reçu de nouveaux chars, qui peuvent transporter beaucoup plus de personnes. Malheureusement, cela signifie aussi que les chars sont trop grands et se retrouvent souvent coincés lorsqu'il y a du trafic en sens inverse. Pour éviter cela, l'Empereur a décidé de rendre toutes les routes de l'Empire à sens unique.

Cependant, l'Empereur aime se rendre assez souvent dans sa ville natale, et il veut s'assurer que, lorsqu'il s'y trouve, il puisse rapidement retourner à Rome si quelque chose d'important survient. Pour cela, il souhaite que le chemin le plus court de sa ville natale à Rome ne devienne pas plus long qu'il ne l'est actuellement. De plus, il veut également pouvoir retourner dans sa ville natale depuis Rome, mais ce trajet n'a pas besoin d'être aussi rapide que possible.

L'Empire romain est constitué de N villes et M routes bi-directionnelles. La ville natale de l'Empereur est la ville 0 et Rome est la ville $N - 1$. Formellement, il veut choisir une direction pour chaque route. Il veut s'assurer que le chemin le plus court de la ville 0 à la ville $N - 1$ ne soit pas plus long qu'il n'était avant que les routes deviennent à sens unique. De plus, il veut s'assurer qu'il y ait encore un chemin de la ville $N - 1$ à la ville 0. Votre mission est d'orienter les routes en respectant ces conditions, ou d'informer l'Empereur que c'est impossible.

Il est garanti que chaque ville de l'Empire romain est atteignable depuis n'importe quelle autre ville. Notez que cela n'a pas forcément à être le cas après que vous ayez orienté les routes.

Notez qu'orienter les routes de telle manière qu'il soit toujours possible d'aller de 0 à $N - 1$ et de $N - 1$ à 0, sans forcément minimiser la longueur d'un des trajets, vous donnera quand même une partie des points.

Implémentation

Vous devez soumettre un seul fichier source `.cpp`.



Parmi les pièces jointes du problème, vous trouverez un squelette `roads.cpp` avec une implémentation d'exemple.

Vous devez implémenter la fonction suivante :

C++

```
vector<pair<int, int>> orient_roads(int N, int M, vector<int> A,
vector<int> B, vector<int> W)
```

- L'entier N représente le nombre de villes.
- L'entier M représente le nombre de routes.
- Les tableaux A , B et W décrivent les routes de l'Empire romain : il y a une route entre la ville $A[i]$ et la ville $B[i]$ de longueur $W[i]$.
- S'il y a une manière d'orienter les routes en respectant les contraintes, la fonction doit retourner un vecteur de paires de longueur M . La i -ème paire $\{F, T\}$ signifie qu'il y a une route orientée de F vers T .
- S'il n'y a aucune manière d'orienter les routes, la fonction doit retourner un vecteur vide.

Il est garanti qu'il existe au plus une route entre deux villes et que pour chaque i on a $A[i] \neq B[i]$. Si la fonction ne renvoie pas un vecteur de taille 0 ou M , vous obtiendrez le verdict « wrong answer ». Toutes les routes présentes dans l'entrée doivent être présentes dans le vecteur de sortie exactement une fois et aucune autre route ne doit être présente. L'ordre des routes dans le vecteur de sortie n'importe pas.

Évaluateur d'exemple

Parmi les fichiers joints à cet énoncé, vous trouverez une version simplifiée du grader utilisé lors de l'évaluation. Vous pouvez l'utiliser pour tester vos solutions en local. Ce grader lit les données depuis `stdin`, appelle la fonction `orient_roads` et écrit le résultat dans `stdout` selon le format suivant.

L'entrée est composée de $M + 1$ lignes, contenant :

- Ligne 1 : les entiers N et M .
- Ligne $2 + i$ ($0 \leq i < M$) : les entiers A_i , B_i et W_i .

La sortie est constituée de 1 ou $M + 1$ lignes, contenant :

- Ligne 1 : **Yes** ou **No**, indiquant s'il est possible d'orienter les routes en respectant les contraintes.
- Si la réponse est **Yes** : Ligne $2 + i$ ($0 \leq i < M$) : F_i et T_i , indiquant que la route entre ces deux villes est orientée de F_i à T_i .

Dans la sortie, les routes orientées sont affichées dans l'ordre dans lequel elles sont retournées par la fonction.

Contraintes

- $3 \leq N \leq 100\,000$.
- $N - 1 \leq M \leq 100\,000$.
- $0 \leq A_i, B_i \leq N - 1$ et $A_i \neq B_i$.
- $1 \leq W_i \leq 1\,000\,000\,000$.
- Il est garanti que chaque ville de l'Empire romain est atteignable depuis n'importe quelle autre ville.

Score

Votre programme sera testé sur plusieurs tests, regroupés en sous-tâches. Pour obtenir les points d'une sous-tâche, votre programme doit résoudre correctement tous les tests qui lui sont associés.

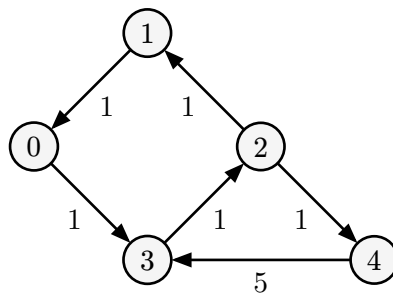
- **Sous-tâche 0 [0 points]**: Exemples.
- **Sous-tâche 1 [6 points]**: $\max(N, M) \leq 20$.
- **Sous-tâche 2 [11 points]**: $M \leq N$.
- **Sous-tâche 3 [17 points]**: Il est garanti qu'il existe une route directe entre la ville 0 et la ville $N - 1$.
- **Sous-tâche 4 [18 points]**: $\max(N, M) \leq 2000$.
- **Sous-tâche 5 [17 points]**: $W_i = 1$.
- **Sous-tâche 6 [31 points]**: Pas de contraintes additionnelles.

⇒ Vous pouvez obtenir la moitié des points d'un test si le chemin le plus court de 0 à $N - 1$ après avoir orienté les routes n'a pas la même longueur que le chemin le plus court de 0 à $N - 1$ avant d'avoir orienté les routes, même s'il n'y avait aucune orientation qui rendait l'Empereur complètement heureux.

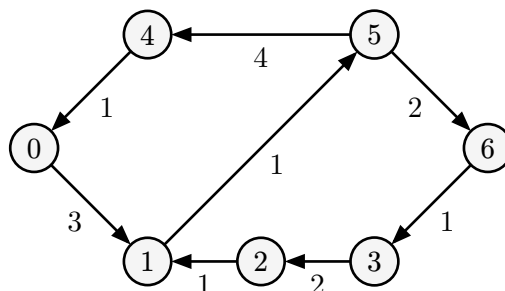
Exemples

stdin	stdout
5 6 0 1 1 1 2 1 0 3 1 3 2 1 2 4 1 3 4 5	Yes 1 0 2 1 0 3 3 2 2 4 4 3
7 8 0 1 3 1 2 1 2 3 2 3 6 1 0 4 1 4 5 4 5 6 2 1 5 1	Yes 0 1 2 1 3 2 6 3 4 0 5 4 5 6 1 5
8 9 0 1 3 1 2 4 2 4 2 0 3 6 3 4 1 4 5 3 5 6 1 6 7 2 5 7 3	No

Explication



Dans le premier exemple, on peut aller de 0 à 4 via $0 \rightarrow 3 \rightarrow 2 \rightarrow 4$. C'est un chemin de longueur 3, qui était aussi la longueur minimale avant que les routes soient orientées. On peut aussi revenir par le chemin $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$. La longueur de ce chemin n'importe pas.



Dans le deuxième exemple, on peut aller de 0 à 6 avec le chemin $0 \rightarrow 1 \rightarrow 5 \rightarrow 6$ et on peut revenir avec $6 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 5 \rightarrow 4 \rightarrow 0$.

Dans le troisième exemple, on peut montrer qu'il n'y a pas d'orientation des routes qui rendrait l'Empereur heureux.